Contents

Linux — Practical Exercises **1**
Exercise 1: System Performance Baseline Script 1
Exercise 2: Log Analyzer 2
Exercise 3: User & Permission Audit Script 2
Exercise 4: Namespace & Cgroup Exploration 3
Exercise 5: Performance Troubleshooting Simulation 3
Exercise 6: systemd Service Creation 4
Checkpoint Questions 5

Linux — Practical Exercises

Exercise 1: System Performance Baseline Script

Goal: Create a script that collects system performance metrics and generates a report.

```
#!/usr/bin/env bash
# system-report.sh — Run and analyze output
set -euo pipefail

OUTPUT="/tmp/sysreport-$(date +%Y%m%d-%H%M%S).txt"

echo "=== System Performance Report ===" > "$OUTPUT"
echo "Date: $(date)" >> "$OUTPUT"
echo "" >> "$OUTPUT"

echo "--- CPU Info ---" >> "$OUTPUT"
lscpu | grep -E "Model name|CPU\(s\)|Thread|Socket" >> "$OUTPUT"

echo "--- Memory ---" >> "$OUTPUT"
free -h >> "$OUTPUT"

echo "--- Disk Usage ---" >> "$OUTPUT"
df -h >> "$OUTPUT"

echo "--- Top 10 CPU Consumers ---" >> "$OUTPUT"
ps aux --sort=-%cpu | head -11 >> "$OUTPUT"

echo "--- Top 10 Memory Consumers ---" >> "$OUTPUT"
ps aux --sort=-%mem | head -11 >> "$OUTPUT"

echo "--- Network Connections ---" >> "$OUTPUT"
ss -s >> "$OUTPUT"

echo "--- Load Average ---" >> "$OUTPUT"
cat /proc/loadavg >> "$OUTPUT"

echo "--- Inode Usage ---" >> "$OUTPUT"
df -i | awk 'NR==1 || $5+0 > 50' >> "$OUTPUT"

cat "$OUTPUT"
echo "Report saved to $OUTPUT"
```

Tasks: 1. Run the script and analyze the output 2. Add disk I/O stats using `iostat` 3. Add detection of zombie processes 4. Schedule it via cron every hour

---

Exercise 2: Log Analyzer

Goal: Parse nginx access logs and extract top endpoints, error rates, and slow requests.

```
#!/usr/bin/env bash
# log-analyzer.sh
LOG_FILE="${1:-/var/log/nginx/access.log}"

if [[ ! -f "$LOG_FILE" ]]; then
    echo "Usage: $0 /path/to/access.log"
    exit 1
fi

echo "=== Log Analysis: $LOG_FILE ==="
echo "Total requests: $(wc -l < "$LOG_FILE")"
echo ""

echo "--- Top 10 URLs ---"
awk '{print $7}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- HTTP Status Code Distribution ---"
awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -rn

echo ""
echo "--- Top 10 Client IPs ---"
awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- 5xx Errors ---"
awk '$9 >= 500 {print $7, $9}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

echo ""
echo "--- Requests > 2 seconds (if response time available) ---"
awk '{if ($NF+0 > 2) print $0}' "$LOG_FILE" | wc -l
```

Tasks: 1. Add time-based filtering (last hour only) 2. Generate CSV output for Excel/Grafana import 3. Add alert if error rate > 5%

Exercise 3: User & Permission Audit Script

Goal: Audit system for security misconfigurations.

```
#!/usr/bin/env bash
# security-audit.sh
set -euo pipefail

echo "=== Security Audit Report ==="

echo ""
echo "--- Users with UID 0 (root-equivalent) ---"
awk -F: '$3==0 {print $1}' /etc/passwd

echo ""
echo "--- Users with empty passwords ---"
awk -F: '$2=="" {print $1}' /etc/shadow 2>/dev/null || echo "Need root"

echo ""
echo "--- SUID binaries ---"
find / -perm -4000 -type f 2>/dev/null | head -20
```

```
echo ""
echo "--- SGID binaries ---"
find / -perm -2000 -type f 2>/dev/null | head -20

echo ""
echo "--- World-writable files in /etc ---"
find /etc -perm -o+w -type f 2>/dev/null

echo ""
echo "--- Listening services ---"
ss -tlnp

echo ""
echo "--- sudo rules ---"
cat /etc/sudoers 2>/dev/null | grep -v '^#' | grep -v '^$'
```

Exercise 4: Namespace & Cgroup Exploration

Goal: Understand how containers work at the kernel level.

```
# 1. Create an isolated process using unshare
sudo unshare --fork --pid --mount-proc bash
# Now inside isolated PID namespace
ps aux     # Should only show bash and ps
exit

# 2. Explore a container's namespaces
docker run -d --name test nginx
CONTAINER_PID=$(docker inspect -f '{{.State.Pid}}' test)
ls -la /proc/$CONTAINER_PID/ns/

# 3. Enter container network namespace from host
sudo nsenter -t $CONTAINER_PID --net ip addr

# 4. Check cgroup limits
cat /sys/fs/cgroup/memory/docker/$(docker inspect -f '{{.Id}}' test)/memory.limit_in_bytes

# 5. Create your own cgroup
sudo mkdir /sys/fs/cgroup/cpu/demo
echo 25000 | sudo tee /sys/fs/cgroup/cpu/demo/cpu.cfs_quota_us   # 25% CPU
echo 100000 | sudo tee /sys/fs/cgroup/cpu/demo/cpu.cfs_period_us  # period
# Add PID to cgroup
echo $$ | sudo tee /sys/fs/cgroup/cpu/demo/cgroup.procs
```

Exercise 5: Performance Troubleshooting Simulation

Goal: Diagnose simulated performance issues.

```
# Simulate CPU stress
stress-ng --cpu 2 --timeout 60 &

# Observe with these tools simultaneously:
# Terminal 1: top -H -p $(pgrep stress-ng | head -1)
# Terminal 2: mpstat -P ALL 1
# Terminal 3: vmstat 1
```

```
# Simulate memory leak
cat /dev/urandom | head -c 500M > /tmp/bigfile &

# Observe:
# free -h  (watch changes)
# cat /proc/meminfo | grep -E "MemFree|Cached|Buffers"

# Simulate disk I/O
dd if=/dev/zero of=/tmp/testfile bs=1M count=1024 oflag=dsync &
# iostat -xz 1

# Clean up
rm /tmp/bigfile /tmp/testfile
killall stress-ng 2>/dev/null
```

Exercise 6: systemd Service Creation

Goal: Create and manage a custom systemd service.

```
# 1. Create a simple Python HTTP server script
cat > /usr/local/bin/myapp.sh << 'EOF'
#!/bin/bash
python3 -m http.server 8888 --directory /tmp
EOF
chmod +x /usr/local/bin/myapp.sh

# 2. Create systemd unit file
cat > /etc/systemd/system/myapp.service << 'EOF'
[Unit]
Description=My Test HTTP Server
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=nobody
ExecStart=/usr/local/bin/myapp.sh
Restart=on-failure
RestartSec=5s
StandardOutput=journal
StandardError=journal
Environment="PORT=8888"
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
EOF

# 3. Enable and start
systemctl daemon-reload
systemctl enable myapp
systemctl start myapp
systemctl status myapp
curl localhost:8888

# 4. Test restart behavior
kill $(pgrep -f "http.server 8888")
sleep 6
```

```
systemctl status myapp  # Should have restarted

# 5. View logs
journalctl -u myapp -n 50 --no-pager
```

Checkpoint Questions

After completing exercises, answer these:

1. What happened to the cgroup limits when you ran stress-ng?
2. Which files were open in the nginx container's PID namespace?
3. How did systemd behave when you killed the process?
4. What was the most memory-consuming process during the memory simulation?
5. What SUID binaries did the security audit find, and are they expected?